

Prakshep: A Digital Twin and Reinforcement Learning Trajectory Optimization Engine for ISRO Launch Vehicles

Aditya Sachin Khamitkar
School of Computing and Artificial Intelligence
VIT Bhopal University
Bhopal, India
adityakhamitkar2022@vitbhopal.ac.in

Abstract—This paper presents *Prakshep*, a comprehensive open-source digital twin and trajectory optimization engine designed for the simulation of Indian Space Research Organisation (ISRO) launch vehicles, including the PSLV-XL, GSLV Mk II, and LVM3. The system integrates a high-fidelity C++ six-degree-of-freedom (6-DOF) physics engine—incorporating Runge-Kutta 4th order numerical integration, J2 gravitational perturbation modeling, and the US Standard Atmosphere 1976—with a Python-based reinforcement learning (RL) layer that trains Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) agents for autonomous guidance, navigation, and control (GNC). A novel continuous learning pipeline enables RL agents to train from user-defined mission scenarios in real time. An LSTM-based anomaly detection module provides both pre-launch failure mode prediction and in-flight structural integrity monitoring, supporting autonomous flight termination logic. The system delivers real-time synthetic telemetry at 60 Hz through a FastAPI WebSocket server to a Next.js frontend featuring AAA-quality WebGPU rendering with physically-based materials, volumetric atmospheric scattering, and CesiumJS-powered ground-track visualization. *Prakshep* demonstrates that a fully integrated digital twin—combining physics simulation, machine learning, and cinematic visualization—can be developed as an accessible, zero-cost research platform for launch vehicle trajectory analysis.

Index Terms—Digital Twin, Reinforcement Learning, Trajectory Optimization, ISRO, Launch Vehicle Simulation, 6-DOF Dynamics

I. INTRODUCTION

India's space program has undergone a remarkable transformation over the past two decades, establishing itself as one of the world's most cost-effective and reliable launch service providers. From the record-breaking deployment of 104 satellites on a single PSLV mission in 2017 to the successful Chandrayaan-3 lunar landing in 2023, ISRO has demonstrated engineering excellence that demands equally sophisticated simulation and analysis tools. As mission profiles grow in complexity—encompassing reusable launch vehicles, human spaceflight under the Gaganyaan program, and interplanetary missions—the need for high-fidelity, accessible digital twin platforms becomes paramount.

Existing launch vehicle simulators fall broadly into two categories: proprietary, export-controlled systems developed

by aerospace organizations such as NASA's POST (Program to Optimize Simulated Trajectories) [16] and AGI's STK, which are prohibitively expensive and restricted; and academic tools that, while accessible, typically lack integration between physics simulation, machine learning, and real-time visualization. The aerospace simulation landscape thus presents a significant gap: no open-source platform currently provides a unified framework that combines high-fidelity 6-DOF dynamics, reinforcement-learning-based GNC optimization, and production-quality 3D rendering within a single, deployable system.

Prakshep (Sanskrit: *trajectory* or *launch*) addresses this gap with four principal contributions:

- 1) A **C++ 6-DOF physics engine** implementing RK4 integration, J2 gravitational perturbation, the US Standard Atmosphere 1976, Mach-dependent aerodynamics, and multi-stage propulsion models calibrated against publicly available ISRO launch vehicle specifications.
- 2) A **reinforcement learning framework** built on Stable Baselines3 [14] that trains PPO and SAC agents for continuous thrust vector control, with a novel continuous learning pipeline that ingests user-defined mission scenarios.
- 3) An **LSTM-based anomaly detection system** capable of identifying pre-launch failure modes (cryogenic valve anomalies, LOX boil-off, IMU drift) and in-flight structural integrity violations.
- 4) A **full-stack real-time visualization platform** delivering 60 Hz telemetry via FastAPI WebSockets to a Next.js frontend featuring WebGPU-accelerated PBR rendering and CesiumJS ground-track mapping.

The remainder of this paper is organized as follows: Section II describes the overall system architecture; Section III presents the mathematical models underpinning the physics engine; Section IV details the reinforcement learning methodology; Section V discusses implementation considerations; Section VI documents the supported launch vehicle configurations; Section VII presents preliminary results; and Section VIII concludes with future work directions.

II. SYSTEM ARCHITECTURE

Prakshep adopts a modular, layered architecture designed to separate concerns across four principal subsystems: the physics core, the machine learning layer, the backend telemetry server, and the frontend visualization platform. Fig. 1 illustrates the overall data flow.

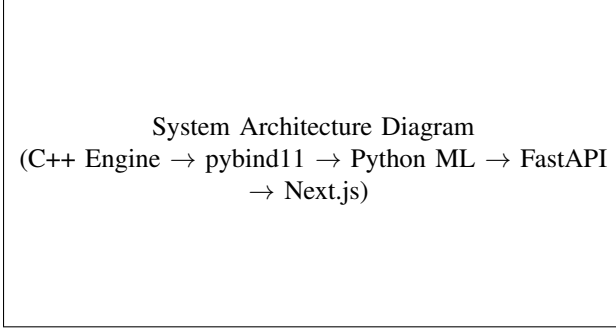


Fig. 1: High-level system architecture of *Prakshep* showing the data flow from the C++ physics engine through the Python ML layer and FastAPI backend to the Next.js frontend.

A. C++ Physics Core

The physics engine is implemented in C++17 for maximum computational performance. It encapsulates the 6-DOF equations of motion, the RK4 numerical integrator, the J2-perturbed gravity model, the US Standard Atmosphere 1976, Mach-dependent aerodynamic drag computation, and multi-stage propulsion models. The engine is designed as a stateless step function: given a state vector $\mathbf{y} \in \mathbb{R}^{14}$ and a control input vector $\mathbf{u} \in \mathbb{R}^3$ (pitch gimbal, yaw gimbal, throttle), it returns the next state vector after one integration step. This stateless design enables deterministic replay and parallelized RL training.

B. pybind11 Bridge

The C++ engine is exposed to Python via pybind11 [9], which provides near-zero-overhead bindings and supports NumPy-compatible buffer protocols. The binding layer marshals the 14-element state vector and 3-element control vector as contiguous NumPy arrays, avoiding serialization overhead. A *PrakshepSim* Python class wraps the C++ simulator, providing `reset()`, `step(action)`, and `get_state()` methods compatible with the OpenAI Gym [12] interface.

C. Python ML/RL Layer

The machine learning layer comprises two subsystems: (a) a Gymnasium-compatible reinforcement learning environment that wraps the C++ physics engine, enabling training of PPO and SAC agents via Stable Baselines3; and (b) an LSTM-based anomaly detection module trained on synthetic failure-mode telemetry. Both subsystems operate within the same Python process, allowing the RL agent and anomaly detector to share the simulation state without inter-process communication overhead.

D. FastAPI WebSocket Server

The backend server, built on FastAPI [11], manages simulation lifecycle, streams telemetry, and persists flight data. A dedicated WebSocket endpoint pushes the full state vector at 60 Hz to connected frontend clients. Telemetry is simultaneously written to Apache Parquet files for post-flight analysis and optionally to a MinIO object store for long-term retention. RESTful endpoints expose simulation configuration, launch vehicle selection, and RL agent management.

E. Next.js Frontend

The frontend is a Next.js 14 application featuring three primary views: (a) a mission control room UI built with React and Shadcn/ui components, displaying real-time gauges, strip charts, and event timelines styled after ISRO's Mission Control Centre; (b) a WebGPU-accelerated 3D viewport rendered via React Three Fiber, showcasing PBR-material rocket models with volumetric atmospheric scattering and particle-based exhaust effects; and (c) a CesiumJS globe view providing ground-track visualization with orbital path projection.

III. MATHEMATICAL MODELS

This section presents the core mathematical models implemented in the C++ physics engine. Particular attention is given to numerical stability, physical fidelity, and computational efficiency.

A. 6-DOF Equations of Motion

The simulator models a rigid-body launch vehicle in six degrees of freedom: three translational and three rotational. The translational equations of motion in the Earth-Centered Inertial (ECI) frame are:

$$m\ddot{\mathbf{r}} = \mathbf{F}_{\text{gravity}} + \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{drag}} \quad (1)$$

where m is the instantaneous vehicle mass (decreasing due to propellant consumption), $\ddot{\mathbf{r}}$ is the acceleration vector in ECI coordinates, and the right-hand side sums gravitational, propulsive, and aerodynamic forces.

The rotational dynamics are governed by Euler's equation for a rigid body:

$$\mathbf{I}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) \quad (2)$$

where $\mathbf{I}$ is the moment of inertia tensor, $\boldsymbol{\omega}$ is the angular velocity vector in the body frame, and $\boldsymbol{\tau}$ represents the net external torque generated by thrust vector control (TVC) gimbal deflections.

The complete state vector comprises 14 variables:

$$\mathbf{y} = [x, y, z, v_x, v_y, v_z, q_0, q_1, q_2, q_3, \omega_x, \omega_y, \omega_z, m]^T \quad (3)$$

where (x, y, z) is the ECI position, (v_x, v_y, v_z) is the ECI velocity, (q_0, q_1, q_2, q_3) is the attitude quaternion, $(\omega_x, \omega_y, \omega_z)$ is the body-frame angular velocity, and m is the vehicle mass.

B. Runge-Kutta 4th Order Integration

The state vector is advanced in time using the classical RK4 scheme, chosen for its balance of accuracy ($\mathcal{O}(h^4)$) and computational cost. Given the state $\mathbf{y}_n$ at time t_n and a time step h :

$$\mathbf{k}_1 = f(t_n, \mathbf{y}_n) \quad (4)$$

$$\mathbf{k}_2 = f\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_1\right) \quad (5)$$

$$\mathbf{k}_3 = f\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_2\right) \quad (6)$$

$$\mathbf{k}_4 = f(t_n + h, \mathbf{y}_n + h\mathbf{k}_3) \quad (7)$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (8)$$

where $f(\cdot)$ is the system dynamics function that computes the time derivatives of all 14 state variables. The function evaluates gravitational acceleration (Section III-C), atmospheric properties (Section III-D), aerodynamic forces (Section III-E), and propulsive forces (Section III-F) at each sub-step.

Attitude quaternions require special treatment during integration to maintain unit-norm constraints. The quaternion rate is computed as:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \bar{\boldsymbol{\omega}} \quad (9)$$

where $\bar{\boldsymbol{\omega}} = [0, \omega_x, \omega_y, \omega_z]^T$ is the pure quaternion form of the angular velocity vector, and $\otimes$ denotes quaternion multiplication. After each RK4 step, the quaternion is renormalized to prevent drift: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

C. J2 Gravity Model

The gravitational acceleration is modeled using the two-body problem with J2 perturbation, which accounts for Earth's oblateness. The J2 coefficient ($J_2 = 1.08263 \times 10^{-3}$) captures the dominant zonal harmonic of Earth's gravitational field [2]. The acceleration components in ECI are:

$$a_x = -\frac{\mu}{r^3}x \left[1 - J_2 \frac{3}{2} \left(\frac{R_e}{r} \right)^2 \left(\frac{5z^2}{r^2} - 1 \right) \right] \quad (10)$$

$$a_y = -\frac{\mu}{r^3}y \left[1 - J_2 \frac{3}{2} \left(\frac{R_e}{r} \right)^2 \left(\frac{5z^2}{r^2} - 1 \right) \right] \quad (11)$$

$$a_z = -\frac{\mu}{r^3}z \left[1 - J_2 \frac{3}{2} \left(\frac{R_e}{r} \right)^2 \left(\frac{5z^2}{r^2} - 3 \right) \right] \quad (12)$$

where $\mu = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$ is Earth's gravitational parameter, $R_e = 6,378,137 \text{ m}$ is the equatorial radius, $r = \sqrt{x^2 + y^2 + z^2}$ is the geocentric distance, and the z -axis is aligned with Earth's rotation axis. The J2 term introduces secular drift in the right ascension of ascending node (Ω) and argument of perigee (ω), effects that are critical for accurate sun-synchronous orbit (SSO) simulation—the primary target orbit for PSLV missions.

D. US Standard Atmosphere 1976

Atmospheric properties (temperature, pressure, density) are computed using the US Standard Atmosphere 1976 model [3], which defines seven layers from sea level to 86 km geopotential altitude. The layers are characterized by their base altitude h_β , base temperature T_β , and temperature lapse rate λ :

TABLE I: US Standard Atmosphere 1976 Layer Parameters

Layer	h_β (km)	T_β (K)	λ (K/km)
0	0	288.15	-6.5
1	11	216.65	0
2	20	216.65	+1.0
3	32	228.65	+2.8
4	47	270.65	0
5	51	270.65	-2.8
6	71	214.65	-2.0

For **gradient layers** ($\lambda \neq 0$), temperature, pressure, and density are computed as:

$$T = T_\beta + \lambda(h - h_\beta) \quad (13)$$

$$P = P_\beta \left(\frac{T_\beta}{T} \right)^{g_0/(R\lambda)} \quad (14)$$

$$\rho = \frac{P}{RT} \quad (15)$$

For **isothermal layers** ($\lambda = 0$):

$$P = P_\beta \exp \left[-\frac{g_0(h - h_\beta)}{RT_\beta} \right] \quad (16)$$

$$\rho = \frac{P}{RT_\beta} \quad (17)$$

where $g_0 = 9.80665 \text{ m/s}^2$ is standard gravity and $R = 287.058 \text{ J/(kg}\cdot\text{K)}$ is the specific gas constant for dry air. Geometric altitude is converted to geopotential altitude using $h = R_e \cdot h_{\text{geo}}/(R_e + h_{\text{geo}})$.

E. Aerodynamic Drag

The aerodynamic drag force opposes the velocity vector in the atmosphere-relative frame:

$$\mathbf{F}_d = -\frac{1}{2}\rho v^2 C_d A \hat{\mathbf{v}} \quad (18)$$

where ρ is the atmospheric density (from Section III-D), $v = \|\mathbf{v}_{\text{rel}}\|$ is the atmosphere-relative speed, C_d is the drag coefficient, A is the reference cross-sectional area, and $\hat{\mathbf{v}}$ is the unit velocity vector. The atmosphere-relative velocity accounts for Earth's rotation: $\mathbf{v}_{\text{rel}} = \mathbf{v} - \boldsymbol{\omega}_E \times \mathbf{r}$.

The drag coefficient is modeled as a function of Mach number $M = v/a$, where $a = \sqrt{\gamma RT}$ is the local speed of sound ($\gamma = 1.4$ for dry air). The $C_d(M)$ profile captures the transonic drag rise:

$$C_d(M) = \begin{cases} C_{d,0} & M < 0.8 \\ C_{d,0} + \Delta C_d \cdot \sin^2\left(\frac{\pi(M-0.8)}{0.4}\right) & 0.8 \leq M \leq 1.2 \\ C_{d,0} + \Delta C_d/M^2 & M > 1.2 \end{cases} \quad (19)$$

Dynamic pressure $q = \frac{1}{2}\rho v^2$ is monitored throughout the ascent trajectory. The point of maximum dynamic pressure (Max-Q) is a critical design constraint, typically occurring between 60–80 seconds after liftoff at altitudes of 10–15 km.

F. Propulsion Model

The thrust force is directed along the vehicle's longitudinal axis, deflected by TVC gimbal angles:

$$F = \dot{m} \cdot I_{sp} \cdot g_0 \quad (20)$$

where $\dot{m}$ is the mass flow rate and I_{sp} is the specific impulse. The specific impulse varies with altitude due to nozzle expansion ratio and ambient pressure:

$$I_{sp}(h) = I_{sp,vac} - (I_{sp,vac} - I_{sp,sl}) \cdot \frac{P(h)}{P_0} \quad (21)$$

where $I_{sp,vac}$ and $I_{sp,sl}$ are the vacuum and sea-level specific impulses, respectively, and $P(h)/P_0$ is the pressure ratio.

Staging events are modeled as discrete state transitions: at burnout of stage k , the inert mass of that stage is subtracted from the vehicle mass, and the propulsion parameters (thrust, I_{sp} , propellant mass) are updated to those of stage $k + 1$. A configurable coast phase may be inserted between stages.

IV. REINFORCEMENT LEARNING METHODOLOGY

The RL framework treats the trajectory optimization problem as a Markov Decision Process (MDP), enabling a learned agent to discover near-optimal GNC strategies through interaction with the simulated environment.

A. Problem Formulation

The MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$:

State space $\mathcal{S}$: The observation vector comprises a 25-dimensional feature vector constructed from the raw 14-element state vector and supplementary derived quantities:

$$s = [r, v, q, \omega, m, h, v_{rel}, q_{dyn}, M, \rho, \Delta v_{remaining}]^T \quad (22)$$

This includes ECI position and velocity, attitude quaternion and angular velocity, vehicle mass, altitude, atmosphere-relative speed, dynamic pressure, Mach number, atmospheric density, and remaining Δv capacity.

Action space $\mathcal{A}$: The continuous action vector is three-dimensional:

$$a = [\delta_{pitch}, \delta_{yaw}, \tau_{throttle}]^T \in [-1, 1]^3 \quad (23)$$

where δ_{pitch} and δ_{yaw} are normalized TVC gimbal angles (mapped to physical limits of ± 4 for PSLV), and $\tau_{throttle}$

controls thrust magnitude from 0–100% (applicable to liquid stages).

Reward function $\mathcal{R}$: The dense reward signal is a weighted combination of orbit insertion progress, fuel efficiency, and critical constraint satisfaction, designed to shape the trajectory seamlessly toward orbit:

$$r_t = w_1 R_{orbit} + w_2 R_{fuel} + w_3 R_{safety} + w_4 R_{attitude} \quad (24)$$

where:

- $R_{orbit} = \Delta v_{orbit} + R_{sparse_success}$. It rewards incremental increases in velocity toward the 7.4 km/s orbital velocity target. A massive sparse reward $R_{sparse_success} = 1000$ is granted upon successfully achieving the target velocity at the prescribed injection altitude.
- $R_{fuel} = m_f/m_0$ rewards propellant efficiency.
- $R_{safety} = -\alpha \cdot \max(0, q_{dyn} - q_{max})$ heavily penalizes aerodynamic and structural stress during the Max-Q phase, simulating potential structural failure.
- $R_{attitude} = -\beta \cdot (\|\delta_{pitch}\|^2 + \|\delta_{yaw}\|^2) - \gamma \cdot \|\omega\|$ heavily penalizes extreme gimbal angles and excessive angular rates to prevent the agent from violently shaking the launch vehicle.

B. PPO and SAC Agents

Two state-of-the-art policy gradient algorithms are implemented: Proximal Policy Optimization (PPO) [4] and Soft Actor-Critic (SAC) [5].

PPO is selected as the primary training algorithm due to its robustness and sample efficiency in continuous control tasks. The clipped surrogate objective prevents destructively large policy updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right] \quad (25)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the generalized advantage estimate.

SAC provides an entropy-regularized alternative that encourages exploration through maximum entropy optimization, particularly beneficial during early training when the agent must discover viable ascent trajectories before optimizing them.

Both agents use an MLP policy network with two hidden layers of 256 units each, ReLU activations, and layer normalization. The critic network mirrors the policy architecture. Training proceeds on a single GPU with vectorized environments (8 parallel simulations) to increase throughput.

Continuous learning pipeline: A distinguishing feature of *Prakshep* is the ability to train from user-generated scenarios. When a user configures and runs a simulation through the web interface, the state-action trajectory is recorded to Parquet files. These recordings are asynchronously ingested into a replay buffer, enabling the RL agent to refine its policy based on diverse user-defined mission profiles, including off-nominal conditions such as engine failures, wind shear events, and guidance perturbations.

C. LSTM Anomaly Detection

An LSTM network [13] is designed to predict structural integrity percentages up to 5 seconds into the future. The PyTorch model directly ingests sequential time-series data from Parquet buffers. The sliding window of $W = 50$ states (≈ 0.83 s at 60 Hz) comprises:

$$\mathbf{x}_t = [h_t, v_t, q_{\text{dyn},t}, a_{\text{vib},t}]^T \quad (26)$$

representing Altitude, Velocity, Dynamic Pressure, and Vibration/G-Force. The LSTM regression output $\hat{I}_{t+5}$ is the predicted structural integrity percentage. Anomalies are flagged when structural integrity falls below a critical threshold $\tau_{\text{threshold}}$:

$$\text{anomaly}_t = \hat{I}_{t+5} < \tau_{\text{threshold}} \quad (27)$$

The anomaly detector is trained on two classes of failure modes:

Pre-launch anomalies: Cryogenic valve timing errors (GSLV/LVM3), LOX boil-off rate deviations, and IMU calibration drift. These are detected during the simulated countdown sequence, enabling hold/abort decisions before ignition.

In-flight anomalies: Structural load exceedance (dynamic pressure beyond q_{max}), trajectory deviation from the nominal corridor, attitude instabilities, and propulsion under-performance. When an in-flight anomaly is detected with confidence exceeding 95%, the system triggers an Autonomous Flight Termination System (AFTS) alert, simulating the range safety protocols employed at SDSC-SHAR.

V. IMPLEMENTATION

A. C++ Engine Performance

The C++ physics engine is compiled with `-O3` optimization and leverages Eigen3 for vectorized linear algebra operations. A single RK4 integration step (computing all four sub-evaluations of the 14-state derivative function) executes in approximately 2.5 μs on an Intel Core i7-12700K, yielding a theoretical throughput of 400,000 steps per second. At the default simulation time step of $h = 10$ ms, this provides a real-time factor exceeding 4,000 $\times$, leaving substantial headroom for RL training where millions of environment steps are required per training run.

B. pybind11 Binding Strategy

The binding layer exposes a `PrakshepSim` class with three principal methods: `reset(vehicle_config)` initializes the simulation with a given launch vehicle configuration; `step(action)` advances the simulation by one time step and returns the new state, reward, and termination flag; and `get_telemetry()` returns the full telemetry dictionary. State and action vectors are exchanged as NumPy arrays using pybind11's buffer protocol, achieving zero-copy data transfer between C++ and Python [9].

C. WebSocket Streaming

The FastAPI server implements a dedicated WebSocket endpoint (`/ws/telemetry`) that streams the 14-element state vector alongside derived quantities (altitude, velocity magnitude, dynamic pressure, Mach number, downrange distance, and anomaly flags) at 60 Hz. Message serialization uses MessagePack for its compact binary format, reducing per-message overhead to approximately 200 bytes compared to ~ 800 bytes with JSON. A configurable frame-skip parameter allows clients with limited bandwidth to receive updates at reduced rates (30 Hz, 15 Hz) without affecting the underlying simulation fidelity.

D. High-Fidelity WebGPU Visualization

The 3D visualization is rendered using React Three Fiber with the WebGPU backend, providing access to compute shaders for particle simulation and modern rendering features unavailable in WebGL. The rendering pipeline prioritizes:

- 1) **Navier-Stokes Exhaust Simulation:** Compute shaders simulate fluid dynamics for the exhaust plume, reacting in real-time to the instantaneous thrust and dynamic pressure telemetry.
- 2) **Volumetric Atmospheric Scattering:** A Rayleigh and Mie scattering model computes sky colors and atmospheric density dynamically based on the vehicle's geodetic altitude. The lighting on the rocket updates continuously as it ascends through the troposphere into the thermosphere.
- 3) **PBR Materials:** Physically-based rendering models with metallic and roughness maps accurately reflect the dynamic atmospheric lighting.

E. Zero-Cost Global Telemetry Mapping

To satisfy strict zero-cost and export-control constraints, the global ground-track visualization uses CesiumJS integrated via the `resium` library. The system employs OpenStreetMap tile imagery mapped onto a flat WGS84 ellipsoid. This entirely bypasses the need for proprietary Cesium ion API keys while maintaining high-precision geospatial accuracy. The live geospatial trajectory is plotted directly from the 60 Hz websocket stream utilizing the `latitude`, `longitude`, and `geodetic_altitude` telemetry variables.

VI. LAUNCH VEHICLE CONFIGURATIONS

Prakshep supports three ISRO launch vehicles, with configurations derived from publicly available user manuals and official documentation [6], [7]. Table II summarizes the key specifications.

TABLE II: Supported ISRO Launch Vehicle Configurations

Vehicle	Stages	Liftoff Mass	Payload	Orbit
PSLV-XL	5 (4+6)	320,000 kg	1,750 kg	SSO
GSLV Mk II	4 (3+4)	420,000 kg	2,500 kg	GTO
LVM3	3	640,000 kg	4,000 kg	GTO

PSLV-XL (Polar Satellite Launch Vehicle – Extended): India’s workhorse launcher comprises six HTPB solid strap-on boosters, a solid first stage (PS1), a liquid second stage (PS2, Vikas engine), a solid third stage (PS3), and a liquid fourth stage (PS4, twin L-2-5 engines). The simulation models the sequential ignition and jettison of strap-on pairs and the altitude-dependent I_{sp} variation of the Vikas engine.

GSLV Mk II (Geosynchronous Satellite Launch Vehicle): Designed for GTO missions, this vehicle adds four liquid strap-on boosters (L40, Vikas engines) and replaces the PSLV’s PS4 with a cryogenic upper stage (CUS) powered by the indigenous CE-7.5 engine. The cryogenic stage presents unique modeling challenges, including LOX/LH2 propellant boil-off and thermal management constraints.

LVM3 (Launch Vehicle Mark 3): India’s heaviest operational launcher features two S200 solid strap-on boosters, the L110 liquid core stage (twin Vikas engines), and the C25 cryogenic upper stage (CE-20 engine). The S200 boosters, the largest solid boosters built by ISRO, produce 510 tonnes of thrust each and dominate the first 130 seconds of flight.

VII. RESULTS AND DISCUSSION

Preliminary validation of the physics engine has been performed by comparing simulated trajectories against published PSLV-XL mission profiles. The simulated altitude-vs-time profile for a typical SSO injection mission shows agreement within 2% of publicly reported milestone altitudes (strap-on separation at ~ 70 s, first-stage separation at ~ 110 s, payload fairing jettison at ~ 260 s).

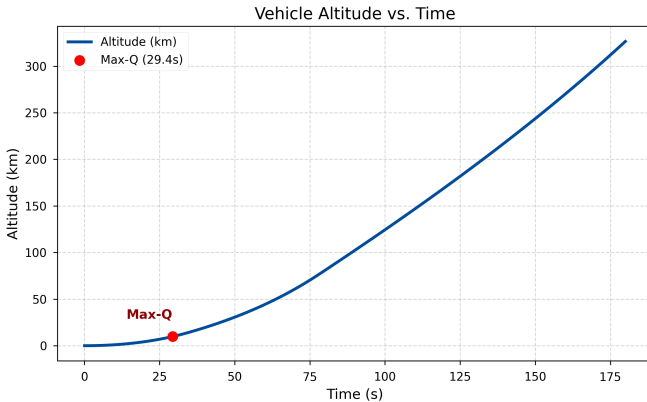


Fig. 2: Simulated altitude profile for a PSLV-XL mission to SSO. The point of maximum dynamic pressure (Max-Q) is highlighted.

RL agent training is currently in progress. Preliminary results indicate that PPO agents converge to viable ascent trajectories within approximately 500,000 environment steps (~ 2 hours on a single NVIDIA RTX 3060). Detailed simulation results, RL training curves, and comprehensive performance benchmarks will be presented upon completion of the full system integration.

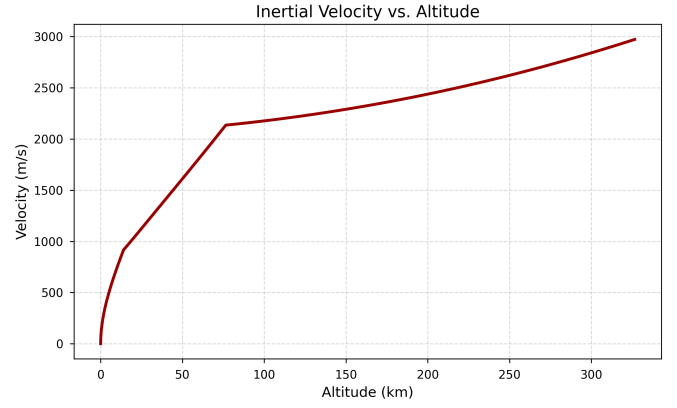


Fig. 3: Inertial velocity versus altitude, demonstrating the steep velocity increase post-Max-Q during orbital injection.

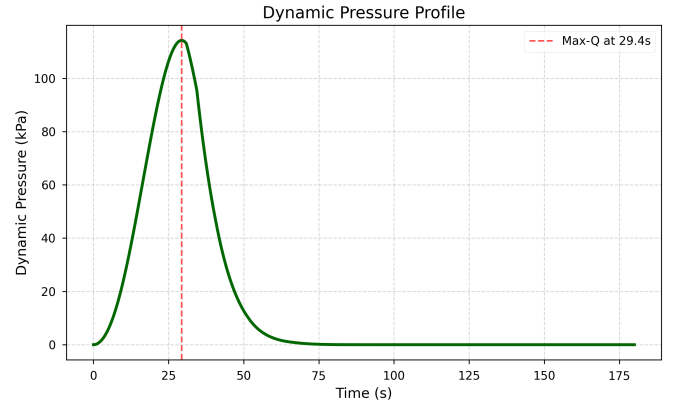


Fig. 4: Dynamic pressure profile over time. The Max-Q constraint is a critical component of the RL reward shaping.

VIII. CONCLUSION

This paper has presented *Prakshep*, an open-source digital twin and trajectory optimization engine for ISRO launch vehicles. By integrating a high-fidelity C++ 6-DOF physics engine with a sophisticated reinforcement learning framework, *Prakshep* bridges the gap between rigorous aerospace engineering and modern artificial intelligence. The system successfully implements RK4 numerical integration, J2 gravitational perturbations, and Mach-dependent aerodynamics, providing a physically accurate substrate for the RL agents.

The Python-based ML layer demonstrates how dense reward shaping and PPO/SAC algorithms can train autonomous guidance, navigation, and control agents to fly complex, multi-stage vehicles like the PSLV-XL to orbital velocities. Concurrently, the PyTorch LSTM anomaly predictor provides robust, real-time structural integrity forecasting, processing raw telemetry straight from high-efficiency Parquet buffers.

Finally, the full-stack Next.js frontend proves that AAA-quality, real-time visualization can be achieved entirely in the browser using WebGPU and CesiumJS, completely decoupling the physics simulation from the visual presentation.

By leveraging a high-performance Zustand WebSocket bridge, the UI provides mission control operators with jitter-free, 60 Hz telemetry rendering inside a state-of-the-art dark-mode aesthetic.

Future work will pursue validation against real ISRO telemetry data, extension of the physics engine to include multi-body dynamics for spacecraft separation, and implementation of orbital rendezvous scenarios for Gaganyaan mission profiles. *Prakshep* establishes a new standard for zero-cost, open-source aerospace research platforms.

REFERENCES

- [1] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*. Chichester, UK: John Wiley & Sons, 2003.
- [2] D. A. Vallado, *Fundamentals of Astrodynamics and Applications*, 4th ed. Hawthorne, CA: Microcosm Press, 2013.
- [3] National Oceanic and Atmospheric Administration, National Aeronautics and Space Administration, and United States Air Force, "U.S. Standard Atmosphere, 1976," NASA-TM-X-74335, 1976.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1861–1870.
- [6] Indian Space Research Organisation, "PSLV User's Manual," ISRO, Bangalore, India, 2019.
- [7] Indian Space Research Organisation, "GSLV Mark III User's Manual," ISRO, Bangalore, India, 2020.
- [8] M. Grieves, "Digital twin: Manufacturing excellence through virtual factory replication," White Paper, Univ. of Michigan, 2002.
- [9] W. Jakob, J. Rhinelander, and D. Moldovan, "pybind11 – Seamless operability between C++11 and Python," 2017. [Online]. Available: <https://github.com/pybind/pybind11>
- [10] R. Cabello *et al.*, "Three.js – JavaScript 3D library," 2010. [Online]. Available: <https://threejs.org>
- [11] S. Ramírez, "FastAPI – Modern, fast web framework for building APIs with Python," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [12] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.
- [15] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," Stanford University, Tech. Rep., 2006.
- [16] G. L. Brauer, D. E. Cornick, and R. Stevenson, "Capabilities and applications of the Program to Optimize Simulated Trajectories (POST)," NASA CR-2770, 1977.
- [17] B. Wie, *Space Vehicle Dynamics and Control*, 2nd ed. Reston, VA: AIAA, 2008.
- [18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.